

Architecture for a Browser-Native 100GB Cloud-Free Peer-to-Peer Transfer Engine

Signaling Infrastructure and Ephemeral Session Negotiation

Browser-to-browser data transfer relies on WebRTC, a technology that requires an out-of-band coordination phase known as signaling¹. Because web browsers reside within private networks and cannot autonomously discover foreign network topologies or resolve public routing addresses across network address translation (NAT) devices, an external transport must mediate the initial exchange of Session Description Protocol (SDP) payloads and Interactive Connectivity Establishment (ICE) candidate metadata¹.

The primary requirement of the signaling layer is strict payload isolation: under no circumstances do file chunks, payload metadata, or storage buffers traverse the signaling mechanism¹. The signaling server functions as a stateless rendezvous conduit.

A connection handshake is initiated through a low-overhead WebSocket transport deployed on distributed edge runtimes, such as Cloudflare Workers with Durable Objects, Node.js with lightweight WebSocket engines, or Go-based microservices⁴. The server maintains an in-memory hash map of active room identifiers paired to sets of active client connections, avoiding external database lookups⁴.

The connection protocol operates through an ephemeral zero-knowledge coordination model⁴. The initiating client generates a cryptographically random, collision-resistant room identifier alongside an ephemeral 256-bit symmetric encryption key⁴.

The symmetric encryption key is embedded exclusively within the Uniform Resource Identifier (URI) hash fragment, structured as

`https://platform.domain/#room=<room_id>&key=<symmetric_key>`⁴. Because user agents do not transmit the URI fragment to the origin server during HTTP requests, the signaling infrastructure remains blind to the decryption key⁴.

The signaling phase executes across a sequence of discrete operational states:

1. **Rendezvous Creation:** The sender registers an ephemeral room record on the signaling server with a hard-coded Time-To-Live (TTL) configured between 60 and 300 seconds¹.
2. **Participant Pairing:** The recipient navigates to the invitation URL, parses the room identifier from the parameter string and the key from the fragment, and opens a WebSocket session to the signaling broker⁴.
3. **Offer and Answer Handshake:** The sender constructs an `RTCPeerConnection` instance, invokes `createOffer()`, sets its local description, and transmits the resulting SDP payload across the WebSocket². The server enforces a strict concurrency ceiling of two peers per room to prevent amplification attacks and forwards the message to the recipient¹. The

recipient processes the offer via `setRemoteDescription()`, generates `createAnswer()`, assigns its local description, and routes the answer back².

4. **Trickle ICE Candidate Streaming:** Rather than delaying the handshake until candidate gathering concludes, peers employ Trickle ICE (RFC 8838)³. As the local browser agent discovers host, server reflexive (STUN), and relayed (TURN) candidate addresses, it pushes them across the signaling channel, allowing route verification to execute concurrently with session description negotiation³.
5. **Immediate Teardown:** Once the `RTCPeerConnection.connectionState` transitions to connected, both endpoints emit an unbind signal and terminate the WebSocket connection⁴. The signaling server purges the room identifier from memory¹. All subsequent application control messages, transfer heartbeats, and retransmission signals are transferred strictly over the peer-to-peer data channel⁵.

Signaling Subsystem Attribute	Technical Specification	Architectural Justification
Transport Layer	Secure WebSockets (wss://) over TLS 1.3	Guarantees bidirectional, low-latency JSON and binary frame exchange through corporate proxies ² .
Room Lifecycle	Ephemeral; 300s TTL or instantaneous peer binding ¹	Eliminates server-side state accumulation and narrows the vulnerability window for room squatting ⁴ .
Peer Concurrency	Strict ceiling: $N =$ endpoints per room namespace ¹	Matches one-to-one transfer semantics; prevents resource exhaustion on signaling nodes ¹ .
Cryptographic Isolation	Client-side URI fragment key encapsulation (#key=...) ⁴	Guarantees that signaling hosts and transit networks cannot intercept session keys ⁴ .
Candidate Gathering	Incremental Trickle ICE (RFC 8838) ³	Minimizes initial Time-To-First-Chunk (TTFC) by overlapping route discovery with peer

		initialization ³ .
--	--	-------------------------------

RTCDataChannel Transport Layer and High-Throughput Optimization

Data transfer occurs entirely within an RTCDataChannel, operating via the Stream Control Transmission Protocol (SCTP) encapsulated over Datagram Transport Layer Security (DTLS), which is layered over the User Datagram Protocol (UDP)³. Achieving multi-gigabit throughput across this stack requires addressing browser-specific buffer allocations, SCTP packet segmentation boundaries, and head-of-line blocking phenomena².

Chunk Sizing and Engine-Specific Ceilings

Modern SCTP implementations in Chromium and Gecko feature data segmentation enhancements, but message payloads cannot be arbitrarily sized¹¹. The WebRTC specification permits large messages, but browser engines enforce rigid limits on individual RTCDataChannel.send() calls:

- Safari and certain Mobile WebKit variants enforce a hard ceiling of **64 KiB (65,536 bytes)** per message frame⁴. Exceeding this boundary triggers an immediate, unrecoverable DOM exception that tears down the data channel⁴.
- Chromium environments permit single calls up to **256 KiB (262,144 bytes)**⁴.
- Transmitting chunks larger than the path Maximum Transmission Unit (MTU, typically **~ 1,200 to 1,400 bytes** post-encapsulation) requires SCTP to segment messages into multiple data chunks¹¹. If an oversized message undergoes mid-stream fragmentation under packet-loss conditions, the entire application-level unit stalls pending reassembly².

The optimal, universally compatible chunk payload size is precisely **64 KiB (65,536 bytes)**⁴. This sizing avoids engine-level message boundary exceptions, minimizes overhead relative to the standard 20-byte SCTP and 13-byte DTLS encapsulation headers, and keeps browser memory copies within optimal CPU cache allocations⁴.

Dual-Threshold Backpressure Management

A common point of failure in browser-based transfers exceeding several gigabytes is outbound buffer saturation⁵. When a client invokes RTCDataChannel.send(), the binary payload is not immediately flushed to the physical network interface; instead, it is enqueued within an internal user-space SCTP buffer queue managed by the browser engine⁵.

If the underlying disk access subsystem or JavaScript event loop reads and enqueues chunks faster than the link's transmission capacity—which is governed by the delay-bandwidth product and congestion window—the RTCDataChannel.bufferedAmount grows unchecked⁵.

When this buffer exceeds system limits (frequently between **16 MB** and **64 MB**

depending on host memory and browser architecture), the tab crashes from an out-of-memory error or silently drops frames⁵.

To resolve this, the sending subsystem implements an explicit dual-threshold backpressure control loop⁵. The high-water mark (H_{wm}) is established at **8 MB** (8,388,608 bytes), and the low-water mark (L_{wm}) is set to **1 MB** (1,048,576 bytes)⁵.

The sender checks `dataChannel.bufferedAmount` during the read loop⁵. If the value exceeds H_{wm} , the read stream suspends chunk extraction from the local disk⁵. The channel's

`bufferedAmountLowThreshold` property is configured to L_{wm} ⁵. The transmission loop generates an asynchronous suspension promise, pausing execution until the underlying engine flushes data over the network and dispatches the native `onbufferedamountlow` event⁵.

Upon trigger, the promise resolves, the event listener unbinds, and slicing resumes until H_{wm} is reached again⁵. This approach ensures that client RAM consumption remains flat at $\leq 16\text{ MB}$ throughout a **100 GB** stream⁵.

Transport Channel Configuration and Head-of-Line Blocking Trade-offs

A foundational design decision involves configuring SCTP reliability and sequencing parameters³. WebRTC defaults to an ordered, fully reliable configuration (`{ ordered: true }`)². In this mode, SCTP mirrors TCP semantics: dropped packets are automatically retransmitted, and segments received out of sequence are retained within the engine buffer until the missing segment arrives².

On networks with high packet loss ($\geq 2\%$) and high round-trip times (RTT), this behavior introduces head-of-line (HOL) blocking, which degrades throughput². The browser engine halts delivery of all subsequent data to the JavaScript application layer until the gap is filled, starving the disk pipeline².

An alternative approach configures the data channel to be unordered and unreliable (`{ ordered: false, maxRetransmits: 0 }`), shifting reliability mechanisms to the application layer⁶. In this mode, every **64 KiB** chunk is wrapped with an 8-byte application sequence index⁴. Chunks flow directly to disk or the reassembly worker without stalling behind dropped packets⁵. The receiver tracks missing chunks via a dynamic bitmap, sending explicit retransmission vectors across an auxiliary control channel.

However, running application-level retransmissions over an unordered channel can destabilize browser-based congestion control. When user-space SCTP has reliability deactivated, it disables its internal congestion avoidance mechanisms (such as RFC 4960 rate limiting), which can saturate the link and lead to bufferbloat or widespread packet loss³.

Therefore, the production architecture employs ordered, fully reliable channels paired with

dual-threshold backpressure as the default transport². Unordered streams with application-level Selective Acknowledgments (SACK) are reserved as a specialized fallback for high-latency, lossy wide-area network paths where TCP-style throughput collapse is observed¹⁰.

Zero-Footprint Browser Storage and Disk Streaming Pipeline

Transferring a **100 GB** file directly inside a standard web browser breaks conventional in-memory paradigms¹. Browsers operate within aggressive sandbox memory limits; a single tab attempting to accumulate multi-gigabyte structures into an in-memory Blob or ArrayBuffer triggers process termination once heap allocation approaches platform ceilings (typically **1.5 to 4 GB** depending on browser bitness and operating system quotas)⁵. A successful architecture must stream data directly to physical persistent storage, reading and writing contiguous byte ranges without accumulating intermediate state in user memory⁵.

File System Access API Versus Origin Private File System

The modern web platform provides two distinct disk storage primitives:

The Native File System Access API (`showSaveFilePicker`) allows the user to select an exact destination path on their local file system via an explicit operating-system-level prompt⁵. It provides a `FileSystemWritableFileStream`, allowing the application to sequentially append chunks directly into the destination file via `writableStream.write(chunk)`⁵. However, this API is available almost exclusively in Chromium-based browsers (Chrome, Edge, Opera) and remains deliberately unsupported in Mozilla Firefox and Apple Safari due to origin sandbox and privacy concerns²⁰.

The Origin Private File System (OPFS), standardized under the Storage Foundation framework, is available across all modern browsers, including Chrome, Edge, Firefox (v111+), and Safari (v15.2+)²⁵. OPFS provides a sandboxed, origin-isolated private directory structure hidden from the user's desktop interface²⁶. Within a Web Worker environment, OPFS exposes the `FileSystemSyncAccessHandle`, which provides synchronous, low-overhead disk read and write primitives (`read()`, `write()`, `flush()`, `truncate()`) that bypass the asynchronous main-thread serialization pipeline²⁵.

Tri-Tiered Cross-Browser Ingestion Engine

To satisfy the functional mandate across Chrome, Firefox, and Safari without exceeding host memory bounds, the ingestion architecture implements a prioritized tri-tiered storage strategy¹:

In Tier 1, designed for Chromium desktop browsers, feature detection checks for `window.showSaveFilePicker`²⁴. The recipient browser prompts the user for a destination handle prior to handshake completion⁵. Incoming binary chunks from the data channel are written sequentially via `FileSystemWritableFileStream`⁵. Upon reaching the final byte, `writableStream.close()` commits the data to disk⁵. Peak memory overhead remains minimal (

< 32 MB) throughout the entire transfer⁵.

In Tier 2, designed for Firefox and Safari desktop environments where `showSaveFilePicker` is absent, the engine falls back to OPFS within a Dedicated Web Worker²³. The worker opens a transient swap file using `await rootDir.getFileHandle(transferId, { create: true })` and obtains an exclusive `FileSystemSyncAccessHandle`²⁵. As data chunks arrive from the network thread, the worker executes synchronous writes directly to storage offsets:

```
handle.write(chunkView, {at : offset})
```

This operation writes directly to disk without garbage collection overhead²⁵. Once the transfer completes and passes integrity checks, the worker creates a synthetic blob URL or pipes the file through a registered Service Worker download interceptor (Response wrapping a `ReadableStream`), prompting the browser's download manager to move the staged OPFS payload into the user's download directory¹⁵.

In Tier 3, designed for memory-constrained Safari or mobile configurations where OPFS quota exhaustion is imminent, the client routes chunks through a local Service Worker¹⁵. The Service Worker registers a custom fetch event listener that intercepts a virtual loopback URL (such as `/virtual-download-stream`)³¹. Incoming chunks are enqueued into a `ReadableStream` controller served as an HTTP response containing a `Content-Disposition: attachment` header²⁰. The browser treats the payload as an inbound network download, writing chunks directly to the system download directory as they arrive from the data channel and bypassing in-memory buffer construction¹⁵.

Storage Subsystem Primitive	Chromium (Chrome, Edge)	Gecko (Firefox)	WebKit (Safari)	Architectural Fit for 100GB Transfers
showSaveFilePicker()	Supported (v86+) ²⁰	Not Supported ²⁰	Not Supported ²⁰	Zero-copy; streams directly to the chosen local directory ⁵ .
OPFS (SyncAccessHandle)	Supported (Worker) ²⁵	Supported (v111+) ²⁵	Supported (v15.2+) ²⁵	High throughput; requires temporary local disk staging space ²⁵ .

Service Worker Stream	Supported ³¹	Supported ³¹	Partial (Session limits) ²⁰	Pipes network chunks into browser download engine ¹⁵ .
In-Memory Blob Slicing	Unsupported (>) ²¹	Unsupported (>) ²¹	Unsupported (>) ¹⁹	Fails due to tab out-of-memory crashes ⁵ .

Storage Quota Negotiation and Verification

Before accepting a file transfer, the recipient browser queries local storage capacities via `navigator.storage.estimate()`²⁸. Modern browsers allocate origin quotas based on available physical drive capacities³⁴. Chromium allocates up to **60%** of total hard drive space to a single origin³⁴. Firefox grants an origin up to **50%** of free drive space (capped at **8 TiB**) if persistent storage is granted, but limits unprompted group usage to **10 GiB**³⁵. Safari enforces a dynamic quota that allows approximately **60%** of total disk space on macOS 14+, but restricts embedded runtimes and untrusted web origins to lower thresholds³⁴. The application invokes `navigator.storage.persist()` before establishing the transport session³⁶. If granted, this permission prevents the browser from evicting OPFS swap data during system storage pressure³⁶. If the calculated quota is lower than the transfer size plus a ten percent safety buffer, the platform falls back to native file picker streaming or aborts before allocating network resources⁵.

Multi-Threaded Concurrency Model via Web Workers

Processing large data streams on the primary JavaScript thread degrades application responsiveness⁵. Reading multi-gigabyte disk segments, slicing binary objects, computing rolling checksums, and handling UI renders concurrently can block the browser event loop⁵. This results in dropped UI frames, unresponsiveness, and watchdog termination of the tab process⁵. To maintain 60 FPS UI performance, the client offloads heavy processing to background execution contexts using Dedicated Web Workers⁵.

Thread Isolation and Engine Compatibility Constraints

Browser compatibility requires careful thread separation⁹:

- Modern Chromium engines support initializing `RTCPeerConnection` and `RTCDataChannel` directly within a Dedicated Worker, but Safari (WebKit) does not support WebRTC APIs in background worker contexts⁹.
- The `FileSystemWritableFileStream` interface is accessible on the main window thread in Chromium, but synchronous access via `FileSystemSyncAccessHandle` is restricted strictly to Web Worker contexts²⁵.

Consequently, the architecture adopts a hybrid multi-threaded topology. The Main Thread manages user input, DOM updates, WebSocket signaling, and the `RTCPeerConnection` lifecycle⁵. It monitors the high- and low-water marks on `dataChannel.bufferedAmount`, signaling the background workers to pause or resume read cycles⁵. The Main Thread also receives decoupled transfer state vectors (bytes transferred, transfer velocity, ETA, and verified chunk indices) from background workers to update the UI at a throttled rate (**10 Hz**)¹. The Sender Worker retains the root File handle acquired via the file picker, executes binary slicing via `file.slice(offset, offset + CHUNK_SIZE)`, computes incremental cryptographic hashes using WebAssembly, and packages data frames⁵. It transfers chunks to the main thread using transferable zero-copy `ArrayBuffer` primitives, detaching memory from the worker context to prevent duplicate allocations. The Receiver Worker, active during OPFS fallback mode, holds the active `FileSystemSyncAccessHandle` for the sandboxed target file²⁵. It ingests incoming transferable binary buffers received from the main thread's data channel onmessage listener and writes them to disk via synchronous system calls (`accessHandle.write(buffer, { at: currentOffset })`)⁵. The worker maintains an in-memory bitset tracking confirmed chunk indices, committing the progress bitmap to IndexedDB every 500 chunks to support state recovery⁴.

Execution Context	Assigned Responsibilities	Inter-Thread Communication	Memory Lifecycle Management
Main Window Thread	UI rendering, WebSocket signaling, WebRTC lifecycle, DataChannel backpressure loops ⁵ .	PostMessage routing to Web Workers; native browser event dispatch ⁵ .	Holds no chunk buffers; routes zero-copy references directly between Workers and DataChannel ⁵ .
Sender Web Worker	Source file segmentation, binary framing encapsulation, WASM hashing, read pacing ⁵ .	Transferable ArrayBuffer message passing to Main Thread ⁵ .	Allocates a sliding window of 64 KiB buffers; leverages WASM linear memory ⁵ .
Receiver Web Worker	Frame demultiplexing, OPFS synchronous disk writing, bitmap	Consumes transferred ArrayBuffer instances from	Flushes chunks to storage immediately; issues explicit

	state persistence ²⁵ .	Main Thread ⁵ .	garbage-collection -friendly dereferences ⁵ .
--	-----------------------------------	----------------------------	--

NAT Traversal, Coturn Provisioning, and Fallback Relaying

A cloud-free, browser-to-browser architecture must traverse diverse network security configurations, NAT types, and enterprise boundary firewalls¹. The interactive connection relies on ICE (RFC 8445), which systematically evaluates candidate routes to select the most direct communication path³.

NAT Traversal Mechanics and Failure Modes

During discovery, the ICE agent queries a Session Traversal Utilities for NAT (STUN) server to discover its public reflexive IP address and port mapping¹. If both peers reside behind Full-Cone, Restricted-Cone, or Port-Restricted Cone NAT implementations, hole-punching succeeds¹. The endpoints establish a direct UDP-to-UDP session through which file data flows locally or over the public WAN, bypassing third-party infrastructure¹. This direct path

accommodates approximately **80 to 85%** of residential internet transfers¹.

Conversely, if either endpoint resides behind a Symmetric NAT—common in mobile 4G/5G carrier networks and enterprise egress setups—the external port assigned by the gateway varies depending on the specific destination IP¹⁰. Under these conditions, direct hole-punching fails, and direct UDP pairing cannot be established. A Traversal Using Relays around NAT (TURN) server must act as an in-flight relay to bridge the connection¹.

Coturn Production Hardening and High-Throughput Provisioning

To handle fallback traffic for **100 GB** transfers without crashing or exhausting relay resources, the Coturn server deployment must be hardened specifically for long-lived, high-bandwidth relay sessions⁴⁴. Standard installations apply restrictive default bandwidth

limits (max-bps=1000000)⁴⁴. For transfers up to **100 GB**, these limits must be removed by assigning max-bps=0, bps-capacity=0, user-quota=0, and total-quota=0, allowing data rates to scale to link capacity¹⁶.

The dynamic allocation port range (min-port=49152 to max-port=65535) must provide sufficient UDP sockets for concurrent transfers⁸. Because each relayed session uses a unique pair of ports, the Coturn server must be deployed using host networking (such as --net=host in Docker environments) to avoid network address translation table bottlenecks within container runtimes⁸.

To prevent server-side request forgery (SSRF) and proxy abuse, the server must explicitly deny loopback, private IPv4, and private IPv6 CIDR blocks using denied-peer-ip directives⁴⁵. Without these access control rules, external users could route traffic through the TURN relay to scan or

attack private infrastructure behind the relay's network boundary⁴⁵.

Dynamic Ephemeral Credential Generation

To prevent unauthorized relay usage while avoiding the overhead of maintaining persistent user databases, the platform implements the ephemeral TURN REST API authentication standard⁴⁸.

The signaling server shares a matching static-auth-secret with Coturn⁴⁵.

When an authorized client initializes an RTCPeerConnection, the signaling server computes a short-lived, HMAC-SHA1 signed token set⁴⁸:

$$\text{Username} = \text{Timestamp} + \text{TTL} + " : " + \text{UserID}$$

$$\text{Password} = \text{Base64}\left(\text{HMAC-SHA1}(\text{static-auth-secret}, \text{Username})\right)$$

The client registers these credentials within the connection's iceServers configuration array⁶. If an unauthorized party intercepts the room token after TTL expiration (typically configured between 1 and 24 hours), Coturn's native authentication module rejects the connection request⁴⁸.

Infrastructure Component	Minimum Provisioning Baseline	Sustained Peak Workload Threshold	Failure Mode and Recovery Pattern
Signaling Cluster	2 Nodes (1 vCPU, 1GB RAM)	10,000 Concurrent WebSockets	Process crash triggers instant client retry with exponential backoff ⁶ .
STUN Discovery	Embedded within Coturn instances	50,000 Binding R	Resolves natively over UDP port 3478; stateless request/reply fallback ⁸ .
Coturn Relay Nodes	Dual 10Gbps Egress Instances	2 × Sustained Bandwidth	Client detects relay disconnection and triggers an ICE restart to an alternate server ² .
Bandwidth	Unmetered 10Gbps	Continuous	Dynamic client

Budgeting	Uplink	100 GB Relay Streams	monitoring shifts user to pause mode if bandwidth caps are reached ¹⁶ .
------------------	--------	-----------------------------	--

Application-Level Framing, Flow Control, and Fault Recovery

Because an raw RTCDataChannel functions as a message delivery stream rather than an operating-system-level byte stream, the platform implements an application-level framing protocol to coordinate chunk indexing, flow control, and session recovery¹⁵.

Binary Frame Specification

Every message transmitted across the data channel contains an explicit 16-byte binary header followed by the file chunk payload⁴. Control frames reuse this framing structure with zero-length payloads.

Field Offset	Field Name	Data Type	Functional Description
Byte 0	Message Type	8-bit Unsigned Integer	Identifies message class (0x01 Data, 0x02 Ack, 0x03 Pause, 0x04 Resume, 0x05 ChecksumQuery, 0x06 ChecksumReply).
Byte 1	Flags / Reserved	8-bit Unsigned Integer	Reserved for compression identifiers, encryption markers, or end-of-stream flags.
Bytes 2–3	File Identifier	16-bit Unsigned Integer	Uniquely identifies target files during multi-file or folder transfers ¹ .

Bytes 4–7	Chunk Index	32-bit Unsigned Integer	Contiguous sequence number ($0 \leq i < 2^{32} -$). Provides address space for transfers up to 281 TB .
Bytes 8–15	Absolute Byte Offset	64-bit Unsigned Big-Endian	Absolute file write location; prevents chunk drift during out-of-order writes in OPFS or the File System Access API ⁵ .
Bytes 16+	Payload Buffer	Raw Binary ArrayBuffer	File data slice, nominally sized at 64 KiB (65,536 bytes) ⁴ .

For a **100 GB** file segmented into **64 KiB** increments, the total chunk count is:

$$N = \left\lceil \frac{100 \times 1024^3}{65,536} \right\rceil = 1,638,400 \text{ chunks}$$

Sliding-Window Flow Control and Selective Retransmission

To prevent receiver memory exhaustion on low-spec hardware (such as when a sender on an unmetered symmetric 10Gbps link overwhelms a receiver writing to a slow disk), the platform implements an application-layer sliding window⁵:

$$\text{Window Size } (W) = \text{Receiver Advertised Window (rawnd)}$$

$$\text{In-Flight Chunks } (I_f) = \text{Sent Index} - \text{Last Acknowledged Index}$$

The sender restricts transmission to satisfy $I_f \leq W$. The receiver acknowledges data using an aggregated Selective Acknowledgment (SACK) vector¹³.

Rather than emitting an acknowledgment frame for every individual **64 KiB** payload (which

can cause channel thrashing), the receiver batches acknowledgments into an explicit range frame every 128 chunks (**8 MB**) or every **100 milliseconds** ¹³. The control payload reports the highest contiguous chunk index successfully written to disk, followed by an array of non-contiguous ranges representing out-of-order arrivals.

The sender tracks in-flight chunks using a circular buffer¹⁶. When a gap is flagged across two successive SACK frames, the missing indices are prioritized for immediate retransmission ahead of the sequential read cursor¹³.

Network Drop Recovery and Checkpoint Resumption

When transient connectivity interruptions occur (such as a Wi-Fi drop or mobile IP handover), the underlying transport fails¹. The architecture handles recovery through two distinct mechanisms:

For transient drops under thirty seconds, the browser detects transport failure via `RTCPeerConnection.oniceconnectionstatechange` transitioning to `disconnected` or `failed`⁷. The application thread reconnects to the signaling cluster and initiates an ICE restart via `peerConnection.createOffer({ iceRestart: true })`².

The data channel is preserved, and state negotiation runs over the renegotiated transport paths without re-initializing the application session⁶. The sender halts transmissions, queries the receiver's last written offset via a `0x05 ChecksumQuery` frame, re-aligns its read cursor, and resumes streaming¹.

For extended disconnects where a user accidentally closes their browser tab or suffers an extended outage, state is recovered using a persistent metadata store¹. Both sender and receiver store an identical session state record within local browser IndexedDB instances, indexed by a shared, deterministic file descriptor hash⁴.

The stored record tracks the file signature, total byte length, chunk sizing, the last contiguous byte written, and a serialized chunk completion bitmap⁴³.

When the user re-opens the transfer URL, the client checks IndexedDB for matching records⁴³. If present, the receiver retrieves the saved bitmap⁴³.

Once signaling establishes a new `RTCPeerConnection`, the receiver sends its bitmap vector across the data channel¹. The sender's Web Worker seeks its local file handle to the missing ranges and resumes data streaming, avoiding the need to re-transmit previously transferred data¹.

End-to-End Cryptographic Security and Streaming Integrity Verification

Because user files travel directly over the public internet and may traverse intermediate Coturn relay nodes, the architecture must guarantee data confidentiality, forward secrecy, and end-to-end integrity¹.

Encryption and Channel Secrecy

WebRTC enforces Datagram Transport Layer Security (DTLS) natively across all communication channels¹. During session setup, the browser generates an ephemeral certificate used to

establish the DTLS handshake, enforcing TLS 1.2 or TLS 1.3 ciphers with elliptic curve Diffie-Hellman ephemeral key exchanges paired with AES-128-GCM or ChaCha20-Poly1305¹⁰. When traffic is routed through a TURN server, the relay functions exclusively at the public transport layer: the TURN software strips the external UDP/TURN framing but cannot decrypt the inner DTLS-SRTP/SCTP payload¹. The data remains fully encrypted between the two browser endpoints, preventing intermediate relay nodes from reading or tampering with file contents¹.

Streaming Cryptographic Checksums: BLAKE3 Versus SHA-256

Verifying the integrity of a **100 GB** file inside a browser sandbox requires high-throughput hashing algorithms¹. Standard SHA-256, accessed through the native Web Crypto API (`crypto.subtle.digest("SHA-256", chunk)`), presents two architectural limitations: First, Web Crypto does not expose an incremental streaming API with `update()` and `digest()` methods. Computing a single aggregate SHA-256 hash across a series of chunks requires assembling the entire payload into a single in-memory buffer, which is blocked by the **100 GB** memory limit⁵. Second, pure JavaScript implementations of SHA-256 process data at roughly **30 to 50 MB/s** on common client hardware, which throttles data transfer rates well below available network bandwidths.

To achieve line-rate validation without memory bloat, the hashing subsystem utilizes BLAKE3 compiled to WebAssembly with SIMD acceleration⁴². BLAKE3 uses a tree-hashing structure that provides two distinct operational benefits:

BLAKE3-WASM achieves hashing rates exceeding **500 MB/s** on standard consumer hardware, preventing CPU bottlenecks during transfers⁴². Furthermore, because BLAKE3 processes data as an open-ended Merkle tree, chunks are hashed incrementally as they are read or written to disk⁴². Each **64 KiB** chunk can be validated against an intermediate tree node, allowing the client to detect data corruption mid-stream rather than waiting for the entire **100 GB** transfer to complete¹:

$$\text{Root Node} = \text{Node}(\text{Left Child Hash} \parallel \text{Right Child Hash})$$

Upon writing the final byte to disk, the receiver's Web Worker queries its BLAKE3 state context for the 32-byte root hash⁵. It sends this value across the control channel inside a `0x06 ChecksumReply` frame.

The sender's worker evaluates the digest against its own calculated root hash⁵. If the digests match, the file is committed to the user's filesystem; if they mismatch, the client uses the Merkle tree path to isolate and re-download only the corrupted sub-ranges¹.

Verification Mechanism	Throughput (WASM/Native)	Memory Footprint	Incremental API Support	Fault Isolation Resolution
Web Crypto SHA-256	~ 150 –	Extreme (100 GB In-Memory Buffer) ⁵	No (Requires full-file buffer)	None; full file re-transfer required ¹ .
JS-Pure SHA-256	~ 30 –	Low (<)	Yes (Via software updates)	Single Chunk (64 KiB); CPU-bound ⁵ .
BLAKE3 (WASM-SIMD)	≥ [cite: 42]	Minimal (< Linear WASM) ⁴²	Native (Continuous stream tree) ⁴²	Merkle Tree Branch Isolation ⁴² .

Architectural Synthesis and System Evolution

Building a cloud-free peer-to-peer transfer platform capable of moving **100 GB** files between standard web browsers requires coordinating multiple browser subsystems into a unified streaming pipeline¹. The system coordinates these browser primitives without relying on centralized file storage infrastructure¹:

The sender initiates the transfer by selecting a file through native system dialogs¹. A Sender Web Worker reads chunks from storage via `Blob.slice()` and passes them through an incremental BLAKE3 hashing pipeline⁵.

The Main Thread manages an `RTCDataChannel` over SCTP/DTLS, regulating outbound flow via a

dual-threshold backpressure loop to keep user-space buffers under **16 MB**³.

NAT traversal is managed by an ICE agent that attempts direct peer-to-peer connections via STUN, falling back to an unthrottled Coturn relay authenticated with short-lived HMAC REST credentials when restrictive firewalls prevent direct hole-punching¹.

The receiver's Web Worker consumes incoming chunks, validates intermediate hashes, and streams data directly to disk using either the File System Access API or the Origin Private File System via synchronous access handles⁵. This design prevents browser out-of-memory errors by avoiding large in-memory buffers⁵.

Application-level binary framing tracks chunks with explicit sequence numbers, managing flow control through sliding-window SACK acknowledgments and maintaining transfer checkpoints in IndexedDB to support resume operations across dropped connections¹.

This architecture provides a scalable foundation for direct browser transfers. As web standards evolve—including broader adoption of WebTransport over HTTP/3 and expanded cross-browser support for direct local disk access—this modular design allows individual transport and storage layers to be upgraded without disrupting the underlying streaming pipeline.

Works cited

1. Untitled document.pdf
2. WebRTC vs WebSocket: Which to Use for Real-Time Features?,
<https://www.videosdk.live/blog/webrtc-vs-websocket>
3. WebRTC - High Performance Browser Networking (O'Reilly),
<https://hpbn.co/webrtc/>
4. Building a Zero-Knowledge P2P Sync Engine with WebRTC and,
<https://medium.com/@Delorean/building-a-zero-knowledge-p2p-sync-engine-with-webrtc-and-cloudflare-durable-objects-73ef632c0e59>
5. How I Built a Zero-Install, Multi-Gigabyte P2P File Transfer Engine in,
<https://dev.to/kingupe/how-i-built-a-zero-install-multi-gigabyte-p2p-file-transfer-engine-in-the-browser-with-webrtc--1doe>
6. p2play-js/p2p-game - NPM,
<https://www.npmjs.com/package/@p2play-js/p2p-game?activeTab=dependents>
7. RTCPeerConnection: createDataChannel() method - Web APIs | MDN,
<https://developer.mozilla.org/en-US/docs/Web/API/RTCPeerConnection/createDataChannel>
8. Fix WebRTC Errors & Connection Issues - OpenPacketLoss™ FAQ,
<https://openpacketloss.com/faq>
9. RTCPeerConnection - Web APIs | MDN,
<https://developer.mozilla.org/en-US/docs/Web/API/RTCPeerConnection>
10. The Crash Course For the networking for web developer. - Medium,
<https://medium.com/@niceperson2110/the-crash-course-for-the-networking-for-web-developer-1109d947a23f>
11. Support sending blob over datachannel [41480941] - WebRTC,
<https://issues.webrtc.org/issues/41480941>
12. How to send message when message size over the limit via,
<https://stackoverflow.com/questions/77816156/how-to-send-message-when-message-size-over-the-limit-via-webrtc-and-libdatachannel>
13. Data channel hangs over a lossy connection · Issue #138 · pion/sctp,
<https://github.com/pion/sctp/issues/138>
14. av-hls — Rust audio library // Lib.rs, <https://lib.rs/crates/av-hls>
15. I built mayo.pizza, a file-transfer site that sends files directly ... - Reddit,
https://www.reddit.com/r/webdev/comments/1vd65re/i_built_mayopizza_a_filetransfer_site_that_sends/
16. WebRTC and Buffers - GetStream.io,
<https://getstream.io/resources/projects/webrtc/advanced/buffers/>
17. LocalBeam — P2P File Sharing - Nishant Dev,

- <https://portfolio-ndev.vercel.app/projects/local-beam>
18. VDO.Ninja SDK — API Reference, <https://sdk.vdo.ninja/docs/api-reference.html>
 19. qrdrop - npm.io, <https://npm.io/package/qrdrop>
 20. How to Create a Downloadable File in the Browser - OpenReplay Blog, <https://blog.openreplay.com/create-downloadable-file-browser/>
 21. Large blob file in Javascript - firefox - Stack Overflow, <https://stackoverflow.com/questions/39253244/large-blob-file-in-javascript>
 22. Window: showSaveFilePicker() method - Web APIs | MDN, <https://developer.mozilla.org/en-US/docs/Web/API/Window/showSaveFilePicker>
 23. List of P2P file sharing tools - GitHub Gist, <https://gist.github.com/SMUsamaShah/fd6e275e44009b72f64d0570256bb3b2>
 24. file saver npm: FileSaver.js Review and Security Notes - Safeguard, <https://safeguard.sh/resources/blog/file-saver-npm-package-review>
 25. Origin private file system - Web APIs | MDN, https://developer.mozilla.org/en-US/docs/Web/API/File_System_API/Origin_private_file_system
 26. sync-opfs — a real synchronous Node.js filesystem in the browser, <https://github.com/componentor/opfs-fs>
 27. VERIDEX Review Studio, <https://discoverytechlab.com/veridex-review-studio/>
 28. Storage API - MDN Web Docs - Mozilla, https://developer.mozilla.org/en-US/docs/Web/API/Storage_API
 29. File System API - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Web/API/File_System_API
 30. Discover Chrome - Chrome for Developers, <https://developer.chrome.com/discover>
 31. Support long running service workers with FetchEvent.respondWith, <https://issues.chromium.org/41293818>
 32. Stream large blob file using StreamSaver.js - Stack Overflow, <https://stackoverflow.com/questions/67776919/stream-large-blob-file-using-streamsaver-js>
 33. A Beginner-friendly Guide on Setting Automatic JavaScript Download, <https://www.bitdegree.org/learn/javascript-download>
 34. Storage for the web | Articles, <https://web.dev/articles/storage-for-the-web>
 35. Storage quotas and eviction criteria - Web APIs - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria
 36. Capabilities | 2020 | The Web Almanac by HTTP Archive, <https://almanac.httparchive.org/en/2020/capabilities>
 37. IndexedDB Max Storage Size Limit - Detailed Best Practices - RxDB, <https://rxdb.info/articles/indexeddb-max-storage-limit.html>
 38. What are IndexedDB's storage limits across different browsers, <https://www.mindstick.com/interview/34337/what-are-indexeddb-s-storage-limits-across-different-browsers-platforms>
 39. RTCPeerConnection() constructor - Web APIs, <https://developer.mozilla.org/en-US/docs/Web/API/RTCPeerConnection/RTCPeer>

Connection

40. WebRTC Peer-to-peer connections | Can I use... Support ... - CanIUse, <https://caniuse.com/rtppeerconnection>
41. dc.send(file) не существует: что на самом деле нужно для, <https://habr.com/ru/articles/1022522/>
42. blake3 1.8.7 - Docs.rs, <https://docs.rs/crate/blake3/latest/source/README.md>
43. File MozillaFirefox.changes of Package MozillaFirefox, https://build.opensuse.org/projects/home:MSirringhaus:webauthn_devel/package/s/MozillaFirefox/files/MozillaFirefox.changes?expand=0
44. Building Real-Time P2P Communication: A Deep Dive into WebRTC, <https://akashsahani2001.medium.com/building-real-time-p2p-communication-a-deep-dive-into-webrtc-ice-stun-and-turn-e645492230c5>
45. Securing coturn: Configuration Guide, <https://www.enablesecurity.com/blog/coturn-security-configuration-guide/>
46. CVE-2025-69217 - Vulnerability-Lookup, <https://vulnerability.circl.lu/vuln/cve-2025-69217>
47. docs/setup/turn/coturn.md · release-v1.97 · Circles / Synapse - GitLab, <https://gitlab.futo.org/circles/synapse/-/blob/release-v1.97/docs/setup/turn/coturn.md>
48. Coturn in Docker: Run a TURN Server with Docker Compose - Meetrix, <https://meetrix.io/blogs/coturn-docker-compose/>
49. CoTURN: How to use TURN REST API? - Stack Overflow, <https://stackoverflow.com/questions/35766382/coturn-how-to-use-turn-rest-api>
50. turnserver · coturn/coturn Wiki - GitHub, <https://github.com/coturn/coturn/wiki/turnserver>
51. turnserver application: a TURN relay server implementation, <https://code.google.com/archive/p/coturn/wikis/turnserver.wiki>
52. pc/peerconnection_integrationtest.cc - external/webrtc - Git at Google, https://chromium.googlesource.com/external/webrtc/+60c8dc8470939fa8cd0ec5414ed805fe01a19606/pc/peerconnection_integrationtest.cc
53. Saving huge files (2GB +) · Issue #163 · eligrey/FileSaver.js - GitHub, <https://github.com/eligrey/FileSaver.js/issues/163>